

```
import sys

class AlphaBox:
    pass

itemA = AlphaBox()
itemA.code_no = 500
itemA.owner = "Khan"

print("1.")
print(itemA.__dict__)

def print_info(reg_no, user_name=None, section_name=None):
    print("\n2.")
    print("ID:", reg_no)

    if user_name:
        print("Name:", user_name)

    if section_name:
        print("Class:", section_name)

print_info(12)
print_info(15, "Hassan", "BSAI")

class BetaRoom:
    pass

print("\n3.")
print(type(BetaRoom))
print(BetaRoom.__dict__.keys())
print(BetaRoom.__module__)

class EmptyOne:
    pass

class EmptyTwo:
    pass

node1 = EmptyOne()
node2 = EmptyTwo()

print("\n4.")
print(isinstance(node1, EmptyOne))
print(isinstance(node2, EmptyTwo))
print(issubclass(EmptyOne, object))
```

```
print(issubclass(EmptyOne, object))
print(issubclass(EmptyTwo, object))

class GradeCard:
    def __init__(self):
        self.user_title = "Areeba"
        self.total_num = 70

sheet = GradeCard()

print("\n5.")
print(sheet.user_title, sheet.total_num)

sheet.user_title = "Noor"
sheet.total_num = 95

print(sheet.user_title, sheet.total_num)

class UserLog:
    def __init__(self):
        self.entry_id = 44
        self.client = "Daniyal"

log1 = UserLog()
log1.batch_no = "C"

print("\n6.")
print(log1.__dict__)

del log1.client

print(log1.__dict__)

class InfoDesk:
    def __init__(self):
        self.num = 7
        self.person = "Raza"
        self.field = "IT"

    def show_all(self):
        for x, y in self.__dict__.items():
            print(x, ":", y)

print("\n7.")
desk1 = InfoDesk()
desk1.show_all()

class DataUnit:
    def __init__(self, a, b, c):
        self.roll = a
```

```

        self.title = b
        self.group = c

print("\n8.")
u1 = DataUnit(1, "Saad", "CS")
u2 = DataUnit(2, "Junaid", "SE")

for rec in [u1, u2]:
    print(rec.__dict__)

print("\n9.")
print(sys.getsizeof(u1))

class SlimUnit:
    __slots__ = ['roll', 'title', 'group']

    def __init__(self, a, b, c):
        self.roll = a
        self.title = b
        self.group = c

mini = SlimUnit(3, "Asad", "AI")

print("\n10.")
print(sys.getsizeof(mini))

```

```

1.
{'code_no': 500, 'owner': 'Khan'}

2.
ID: 12

2.
ID: 15
Name: Hassan
Class: BSAI

3.
<class 'type'>
dict_keys(['__module__', '__dict__', '__weakref__', '__doc__'])
__main__

4.
True
True
True
True

5.
Areeba 70
Noor 95

6.

```

```
{'entry_id': 44, 'client': 'Daniyal', 'batch_no': 'C'}  
{'entry_id': 44, 'batch_no': 'C'}
```

```
7.  
num : 7  
person : Raza  
field : IT
```

```
8.  
{'roll': 1, 'title': 'Saad', 'group': 'CS'}  
{'roll': 2, 'title': 'Junaid', 'group': 'SE'}
```

```
9.  
48
```

```
10.  
56
```

```
class ShapeTrap:  
    def __init__(self, p, q, h):  
        self.p = p  
        self.q = q  
        self.h = h  
  
    def get_area(self):  
        return 0.5 * (self.p + self.q) * self.h
```

```
class ShapePara:  
    def __init__(self, b, h):  
        self.b = b  
        self.h = h  
  
    def get_area(self):  
        return self.b * self.h
```

```
records = [  
    (5, 8, 4, 6, 3),  
    (7, 9, 5, 4, 6),  
    (3, 6, 2, 8, 4)  
]
```

```
step = 1
```

```
for row in records:  
    a1, a2, a3, a4, a5 = row  
  
    fig1 = ShapeTrap(a1, a2, a3)  
    fig2 = ShapePara(a4, a5)
```

```
save_area = {
    "Trapezoid": fig1.get_area(),
    "Parallelogram": fig2.get_area()
}

print("\nSet", step)
print(save_area)

top_one = max(save_area, key=save_area.get)

print("Largest Shape:", top_one)
print("Area:", save_area[top_one])

step += 1
```

```
Set 1
{'Trapezoid': 26.0, 'Parallelogram': 18}
Largest Shape: Trapezoid
Area: 26.0
```

```
Set 2
{'Trapezoid': 40.0, 'Parallelogram': 24}
Largest Shape: Trapezoid
Area: 40.0
```

```
Set 3
{'Trapezoid': 9.0, 'Parallelogram': 32}
Largest Shape: Parallelogram
Area: 32
```

